NU MOA

Week 3: Styling and Layout

FOUNDED FOR FILIPINOS, BUILT FOR THE NATION

Flutter Styling

- Flutter Styling is the process of customizing the appearance of widgets.
- Styling determines how widgets look by modifying colors, fonts, borders, shadows, spacing, and other visual properties.
- Good styling improves readability, usability, and user experience.

Purpose of Styling

- Creates visually appealing interfaces
- Improves readability
- Establishes application branding
- Provides consistency across screens
- Enhances user experience

Common Styling Widgets and Properties

- Flutter provides several ways of style widgets.

Widget / Property	Description
TextStyle	Customizes the appearance of text such as font size, weight, and color.
BoxDecoration	Adds borders, shadows, gradients, and rounded corners to containers.
Container	Applies background colors, sizing, and decorations.
ThemeData	Provides consistent styling throughout the application.
Icon	Displays Material Design icons that can be customized with colors and sizes.

Example: Text Styling

```
child: Text(
  "Hello World",
  style: TextStyle(
    fontSize: 24,
    color: Colors.blue,
    fontWeight: FontWeight.bold,
  ), // TextStyle
), // Text
```

Output

Hello World

Example: Container Styling

```
child: Container(
  width: 200,
  height: 100,
  decoration: BoxDecoration(
    color: Colors.amber,
    borderRadius: BorderRadius.circular(10),
  ), // BoxDecoration
), // Container
```

Output



Flutter Styling

- Flutter Layout refers to the arrangement and positioning of widgets on the screen.
- Layouts determine where widgets are placed and how they adapt to different screen sizes.
- Flutter achieves responsive layouts through nested widgets.

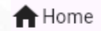
Common Layout Widgets

Widget	Description
Scaffold	Provides the basic page structure.
Row	Arranges widgets horizontally.
Column	Arranges widgets vertically.
Stack	Places widgets on top of one another.
Container	Holds and organizes child widgets.
Padding	Adds internal spacing around widgets.
SizedBox	Creates fixed spacing or sets dimensions.
Expanded	Occupies remaining available space.
Center	Centers a widget within its parent.
Align	Positions widgets within a container.

Example: Row Layout

```
body: Center(
  child: Column(
    children: [
      Text("Username"),
      TextField(),
      ElevatedButton(onPressed: () {}, child: Text("Login")),
    ],
  ), // Column
), // Center
```

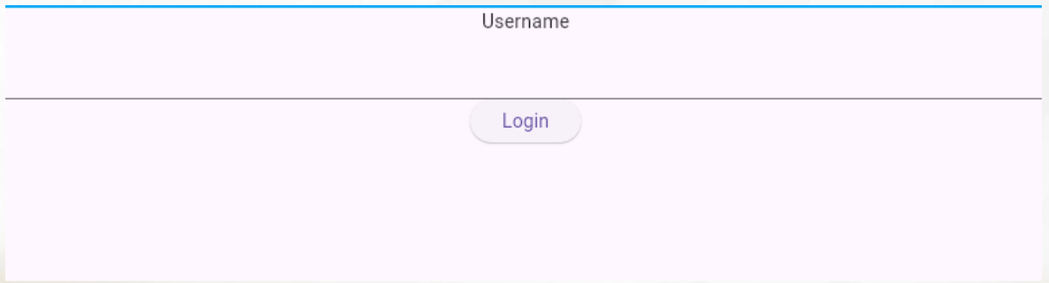
Output



Example: Column

```
body: SizedBox(
  child: Column(
    children: [
      Text("Username"),
      TextField(),
      ElevatedButton(onPressed: () {}, child: Text("Login")),
    ],
  ), // Column
), // SizedBox
```

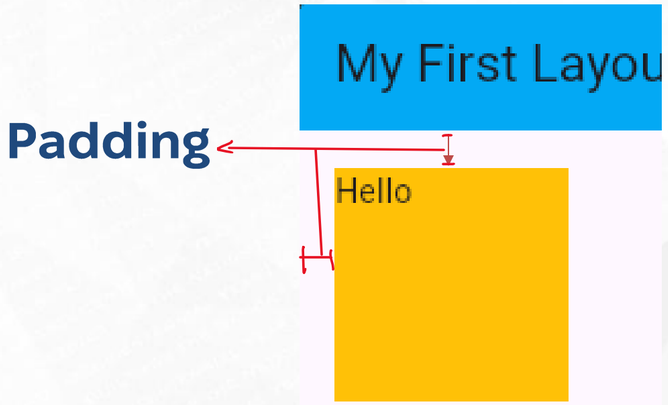
Output



Padding

```
body: SizedBox(
  child: Padding(
    padding: EdgeInsetsGeometry.all(16),
    child: Container(
      color: Colors.amber,
      width: 100,
      height: 100,
      child: Text("Hello"),
    ), // Container
  ) // Padding
), // SizedBox
```

Output



MainAxisAlignment

- Controls the distribution of child widgets along the main axis.
- Examples:

Aligns widgets at the beginning.

```
MainAxisAlignment.start
```

Centers all widgets.

```
MainAxisAlignment.center
```

Aligns widgets at the end.

```
MainAxisAlignment.end
```

MainAxisAlignment

Places equal space around each widget.

```
MainAxisAlignment.spaceAround
```

Places the first and last widgets at the edges.

```
MainAxisAlignment.spaceBetween
```

Distributes equal spacing between all widgets.

```
MainAxisAlignment.spaceEvenly
```

MainAxisAlignment

Places equal space around each widget.

```
MainAxisAlignment.spaceAround
```

Places the first and last widgets at the edges.

```
MainAxisAlignment.spaceBetween
```

Distributes equal spacing between all widgets.

```
MainAxisAlignment.spaceEvenly
```

CrossAxisAlignment

- Controls the alignment of widgets along the cross axis.
- Examples:

```
CrossAxisAlignment.start
```

```
CrossAxisAlignment.center
```

```
CrossAxisAlignment.end
```

```
CrossAxisAlignment.stretch
```

Sample Source Code

```
body: Padding(
  padding: EdgeInsets.all(16),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    spacing: 18,
    children: [
      Text(
        "Flutter Demo",
        style: TextStyle(
          fontSize: 24,
          fontWeight: FontWeight.bold,
          color: Colors.blue,
        ),
      ),

      Container(
        width: double.infinity,
        padding: EdgeInsets.all(16),
        decoration: BoxDecoration(
          color: Colors.amber,
          borderRadius: BorderRadius.circular(10),
        ),
        child: Text(
          "Welcome to Flutter!",
          textAlign: TextAlign.center,
        ),
      ),

      Row(
        mainAxisAlignment: MainAxisAlignment.spaceEvenly,
        children: [
          Icon(Icons.home),
          Icon(Icons.favorite),
          Icon(Icons.settings),
        ],
      ),

      Center(
        child: ElevatedButton(
          onPressed: () {},
          child: Text("Get Started"),
        ),
      ),
    ],
  ),
),
```

Output

